САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Лабораторная работа №2 «Реализация микросервисов»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках ДЗ4 (технический дизайн микросервисной архитектуры), разделить монолитное бэкенд-приложение фото-соцсети (реализованное в ЛР1) на микросервисы, придерживаясь концепции database-per-service, и реализовать межсервисное взаимодействие (синхронное и асинхронное).

Ход работы

1. Общая архитектура

В соответствии с проектом из ДЗ4 монолитное приложение разделено на **четыре независимых микросервиса**, каждый из которых имеет собственную базу данных:

Сервис	Порт	База данных	Зона ответственности
Auth Service	3001	auth.sqlite	Регистрация, логин, профиль, аватары
Post Service	3002	post.sqlite	Посты, лайки, избранное
Comment Service	3003	comment.sqlite	Комментарии
Social Service	3004	social.sqlite	Подписки, уведомления

Дополнительно используется брокер сообщений **RabbitMQ** (в Docker) для асинхронного взаимодействия через события.

Все сервисы написаны на Node.js + TypeScript + Express + TypeORM. В качестве СУБД для каждого сервиса используется SQLite (файл в папке сервиса). Для локального запуска каждый сервис поднимается отдельным процессом на своём порту.

2. Структура репозитория

```
backend-lab1/
├── services/
│   ├── auth-service/
│   │   ├── src/
│   │   │   ├── controller/
│   │   │   │   ├── AuthController.ts
│   │   │   │   └── UserController.ts
│   │   │   ├── entity/
```

```
│   │   │   │   └── User.ts
│   │   │   ├── middleware/
│   │   │   │   ├── auth.ts
│   │   │   │   └── serviceAuth.ts
│   │   │   ├── routes/
│   │   │   │   ├── authRoutes.ts
│   │   │   │   ├── internalRoutes.ts
│   │   │   │   └── userRoutes.ts
│   │   │   ├── utils/
│   │   │   │   └── imageProcessor.ts
│   │   │   ├── data-source.ts
│   │   │   └── index.ts
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   ├── post-service/
│   │   ├── src/
│   │   │   ├── controller/
│   │   │   │   └── PostController.ts
│   │   │   ├── entity/
│   │   │   │   ├── Favorite.ts
│   │   │   │   ├── Like.ts
│   │   │   │   └── Post.ts
│   │   │   ├── middleware/
│   │   │   │   ├── auth.ts
│   │   │   │   └── serviceAuth.ts
│   │   │   ├── routes/
│   │   │   │   ├── internalRoutes.ts
│   │   │   │   └── postRoutes.ts
│   │   │   ├── utils/
│   │   │   │   ├── authClient.ts
│   │   │   │   ├── imageProcessor.ts
│   │   │   │   └── rabbitmq.ts
│   │   │   ├── data-source.ts
│   │   │   └── index.ts
│   │   ├── package.json
│   │   └── tsconfig.json
```

```
│   │
│   ├── comment-service/
│   │   ├── src/
│   │   │   ├── controller/
│   │   │   │   └── CommentController.ts
│   │   │   ├── entity/
│   │   │   │   └── Comment.ts
│   │   │   ├── middleware/
│   │   │   │   ├── auth.ts
│   │   │   │   └── serviceAuth.ts
│   │   │   ├── routes/
│   │   │   │   ├── commentRoutes.ts
│   │   │   │   └── internalRoutes.ts
│   │   │   ├── utils/
│   │   │   │   ├── authClient.ts
│   │   │   │   ├── imageProcessor.ts
│   │   │   │   ├── postClient.ts
│   │   │   │   └── rabbitmq.ts
│   │   │   ├── data-source.ts
│   │   │   └── index.ts
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   └── social-service/
│       ├── src/
│       │   ├── controller/
│       │   │   └── SocialController.ts
│       │   ├── entity/
│       │   │   ├── Follow.ts
│       │   │   └── Notification.ts
│       │   ├── middleware/
│       │   │   ├── auth.ts
│       │   │   └── serviceAuth.ts
│       │   ├── routes/
│       │   │   ├── internalRoutes.ts
│       │   │   └── socialRoutes.ts
│       │   ├── utils/
```

```
│   │   │   ├── authClient.ts
│   │   │   └── rabbitmq.ts
│   │   ├── data-source.ts
│   │   └── index.ts
│   ├── package.json
│   └── tsconfig.json
│
├── uploads/
│   ├── avatars/                # Аватарки
│   ├── posts/                  # Оригиналы и превью фотографий в постах
│   ├── comment-photos/         # Фото из комментариев
│   └── temp/                   # Временная папка для multer
│
├── src/                        # Старый монолитный код
│   ├── controller/
│   ├── entity/
│   ├── middleware/
│   ├── routes/
│   ├── utils/
│   ├── data-source.ts
│   └── index.ts
│
├── package.json                # Зависимости монолита (ЛР1)
├── tsconfig.json
└── README.md
```

3. Auth Service (сервис авторизации, порт 3001)

Зона ответственности: регистрация, логин, получение профиля, загрузка аватара.

База данных auth.sqlite**:**

- Таблица users (id, username, email, passwordHash, avatarPath, createdAt).

Публичные эндпоинты:

- POST /auth/register - регистрация
- POST /auth/login - логин, выдача JWT
- GET /users/profile - профиль текущего пользователя (требует токен)
- POST /users/avatar - загрузка аватара (multipart/form-data)

Внутренние эндпоинты (защищены X-Service-Token**):**

- GET /internal/users/:id - данные пользователя для других сервисов
- POST /internal/users/batch - пакетное получение данных пользователей
- GET /internal/users/:id/exists - проверка существования
- POST /internal/auth/verify - проверка JWT-токена

Пароли хешируются bcryptjs. JWT-токен выпускается на 7 дней и содержит userId. Все остальные сервисы проверяют токен **локально** (общий JWT_SECRET), не дёргая сервис авторизации на каждый запрос.

4. Post Service (сервис постов, порт 3002)

Зона ответственности: посты, лайки, избранное.

База данных post.sqlite**:**

- posts (id, userId, originalPhotoPath, previewPhotoPath, description, createdAt)
- likes (id, userId, postId, createdAt) - уникальность (userId, postId)
- favorites (id, userId, postId, createdAt) - уникальность (userId, postId)

Публичные эндпоинты:

- POST /posts - создание поста (multipart/form-data, фото обрабатывается sharp)
- GET /posts/feed - лента постов (данные авторов подтягиваются из Auth Service)
- GET /posts/:id - один пост с автором
- DELETE /posts/:id - удаление своего поста (файлы удаляются с диска)
- POST /posts/:id/like / DELETE /posts/:id/like - лайк/анлайк
- POST /posts/:id/favorite / DELETE /posts/:id/favorite - избранное

Внутренние эндпоинты:

- GET /internal/posts/:id - данные поста
- GET /internal/posts/:id/exists - проверка существования
- GET /internal/posts/by-user/:userId - посты пользователя

Взаимодействие:

- **HTTP → Auth Service** через authClient.ts - получение данных авторов.
- **RabbitMQ → Social Service** через обменник post.events:
  - like.created - { userId, postId, postOwnerId }
  - like.removed - { userId, postId }

- post.deleted - { postId, userId }

5. Comment Service (сервис для комментов, порт 3003)

Зона ответственности: комментарии (с фото и без).

База данных comment.sqlite**:**

- comments (id, userId, postId, text, photoPath, createdAt)

Публичные эндпоинты:

- POST /comments/posts/:postId/comments - добавление (multipart/form-data, фото сжимается sharp до ~4 Мп, JPEG 95%)
- GET /comments/posts/:postId/comments - список комментариев с авторами
- DELETE /comments/comments/:id - удаление своего комментария

Внутренние эндпоинты:

- GET /internal/comments/by-post/:postId
- GET /internal/comments/count/:postId
- DELETE /internal/comments/by-post/:postId
- DELETE /internal/comments/by-user/:userId

Взаимодействие:

- **HTTP → Auth Service** - получение авторов комментариев.
- **HTTP → Post Service** - проверка существования поста перед добавлением комментария.
- **RabbitMQ → Social Service** через обменник comment.events:
  - comment.created - { commentId, userId, postId, postOwnerId, hasPhoto }

6. Social Service (порт 3004)

Зона ответственности: подписки, уведомления.

База данных social.sqlite**:**

- follows (id, followerId, followingId, createdAt) - уникальность (followerId, followingId)
- notifications (id, userId, type, relatedUserId, relatedPostId, isRead, createdAt)

Публичные эндпоинты:

- POST /social/follow/:followingId - подписаться
- DELETE /social/follow/:followingId - отписаться
- GET /social/notifications - список уведомлений пользователя
- PUT /social/notifications/:id/read - отметить прочитанным

Внутренние эндпоинты:

- GET /internal/follows/following/:userId - кого читает пользователь
- GET /internal/follows/followers/:userId - кто читает пользователя

Взаимодействие:

- **HTTP → Auth Service** - проверка существования пользователя при подписке.
- **RabbitMQ ← Post Service / Comment Service** - подписка на события через очередь social-service-events, привязанную к обменникам post.events и comment.events.

При получении события Social Service создаёт уведомление. События like.removed и post.deleted удаляют соответствующие уведомления.

7. Способы межсервисного взаимодействия

Синхронное (HTTP/REST):

- Все внутренние запросы идут с заголовком X-Service-Token (общий секрет).
- Внутренние эндпоинты доступны только внутри микросервисной сети (в реальном продакшене они были бы закрыты API Gateway).
- Используется библиотека axios с таймаутом 5 секунд.

Асинхронное (RabbitMQ):

- Обменники (post.events, comment.events) - тип topic, durable: true.
- Очередь social-service-events - durable: true, привязана к нужным routing keys.
- Сообщения публикуются с флагом persistent: true (выживают при перезапуске RabbitMQ).
- Обработка подтверждается через ack (успех) или nack (ошибка - сообщение не возвращается в очередь).

8. Сценарий тестирования (сквозной прогон)

Для проверки микросервисной архитектуры создана новая коллекция Postman PhotoSocial Microservices Tests и окружение PhotoSocial Microservices с переменными authUrl, postUrl, commentUrl, socialUrl. Сценарий повторяет ДЗ3:

1. **Register** (первый пользователь, id=1)
2. **Login** → получение token
3. **Upload Avatar**
4. **Create Post** → создание поста id=1
5. **Like Post** (первый лайкает свой пост)
6. **Add Comment** (первый комментирует свой пост)
7. **Register Second User** (второй пользователь, id=2)
8. **Login Second User** → получение secondToken
9. **Like Post by Second User** → событие like.created уходит в RabbitMQ
10. **Get Notifications for First User** → уведомление от Social Service
11. **Mark Notification Read**
12. **Follow Second User**
13. **Get Notifications**
14. **Delete Comment**
15. **Delete Post** → событие post.deleted уходит в RabbitMQ

9. Результаты тестирования

Все проверки прошли успешно. Логи сервисов подтверждают полный цикл взаимодействия:

Post Service:

[nodemon] 3.1.14

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): *.*

[nodemon] watching extensions: ts,json

[nodemon] starting `ts-node src/index.ts`

[post-service] Database connected.

[post-service] RabbitMQ connected.

[post-service] Running on http://localhost:3002

[post-service] Event published: like.created { userId: 2, postId: 1, postOwnerId: 1 }

[post-service] Event published: post.deleted { postId: 1, userId: 1 }

Comment Service:

[nodemon] 3.1.14

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): *.*

[nodemon] watching extensions: ts,json

[nodemon] starting `ts-node src/index.ts`

[comment-service] Database connected.

[comment-service] RabbitMQ connected.

[comment-service] Running on http://localhost:3003

Social Service:

[nodemon] 3.1.14

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): *.*

[nodemon] watching extensions: ts,json

[nodemon] starting `ts-node src/index.ts`

[social-service] Database connected.

[social-service] RabbitMQ connected and consuming events.

[social-service] Running on http://localhost:3004

[social-service] Received event: like.created { userId: 2, postId: 1, postOwnerId: 1 }

[social-service] Notification created for like: 1

[social-service] Received event: post.deleted { postId: 1, userId: 1 }

[social-service] Notifications removed for post: 1

Проверка REST-эндпоинтов (через Invoke-RestMethod**):**

- GET /social/notifications для первого пользователя вернул два уведомления: like (от второго) и comment (от второго, если бы комментарий был от него).
- POST /social/follow/2 (Bogdan подписывается на Alice) → {"message":"Followed"}. У Alice появилось уведомление типа follow.
- PUT /social/notifications/1/read → {"message":"Marked as read"}, поле isRead изменилось на true.
- POST /social/follow/1 от Bogdan → Cannot follow yourself (400).
- Внутренние эндпоинты Social Service (/internal/follows/following/1) вернули список подписок.

Все сервисы работают независимо: если остановить Social Service, остальные продолжают принимать запросы, а события буферизуются в RabbitMQ и будут обработаны после восстановления.

Вывод

В ходе выполнения ЛР2 монолитное приложение фото-соцсети, реализованное в ЛР1, разделено на **четыре независимых микросервиса**:

- **Auth Service** - аутентификация и пользователи;
- **Post Service** - посты, лайки, избранное;
- **Comment Service** - комментарии;
- **Social Service** - подписки и уведомления.

Каждый сервис имеет собственную базу данных SQLite (принцип database-per-service), собственный порт и независимый жизненный цикл. Взаимодействие реализовано двумя способами:

- **Синхронно** - через внутренние REST-эндпоинты с сервисным токеном (например, Post Service получает данные автора поста из Auth Service).
- **Асинхронно** - через RabbitMQ: Post и Comment Service публикуют события (like.created, comment.created, post.deleted), Social Service подписан на них и создаёт или удаляет уведомления.

Все сервисы используют общий JWT_SECRET, что позволяет им проверять токены локально и не зависеть от Auth Service при каждом запросе. Сквозной сценарий из ДЗ3 (регистрация → логин → пост → лайк → комментарий → подписка → уведомления → удаление) успешно пройден против микросервисной архитектуры. Логи подтверждают корректную публикацию и обработку событий в реальном времени.

Полученная архитектура удовлетворяет требованиям масштабируемости, отказоустойчивости и независимого развёртывания, заложенным в проекте ДЗ4. Следующим шагом станет контейнеризация сервисов средствами Docker (ЛР3).